# *Part 2 Building complete applications*

We covered a lot of ground in part 1. We saw how an ASP.NET Core application is composed of middleware, and we focused heavily on minimal API endpoints. We saw how to use them to build JSON APIs, how to extract common code by using filters and route groups, and how to validate input data.

In part 2 we'll dive deeper into the framework, looking at a variety of components that we'll inevitably need to build more complex apps. By the end of this part, you'll be able to build dynamic applications and deploy them to multiple environments, each with a different configuration, saving data to a database.

ASP.NET Core uses dependency injection (DI) throughout its libraries, so it's important that you understand how this design pattern works. In chapter 8 I introduce DI and discuss why it's used. In chapter 9 you'll learn how to configure the services in your applications to use DI.

Chapter 10 looks at the ASP.NET Core configuration system, which lets you pass configuration values to your app from a range of sources: JSON files, environment variables, and many more. You'll learn how to configure your app to use different values depending on the environment in which it's running and how to bind strongly typed objects to your configuration to reduce runtime errors.

In chapter 11 you'll learn how to document your minimal APIs applications by using the OpenAPI specification. Adding an OpenAPI document to your application makes it easier for others to interact with your app, but it has other benefits too. You'll learn how to use Swagger UI to test your app from